

PROJECT

SQL Database Testing — Stock Data Management System

Project objective:

To test the integrity, consistency, and quality of data in a system that stores information about stock markets, companies, daily quotes, and trades.

Tools:

MySQL / phpMyAdmin

Scope of testing:

Data validation, table relationships, business logic, data quality testing, CRUD operations testing.

TEST CASE 1

Title: Verification of company-to-market relationships

Objective: Check whether each company is assigned to an existing market.

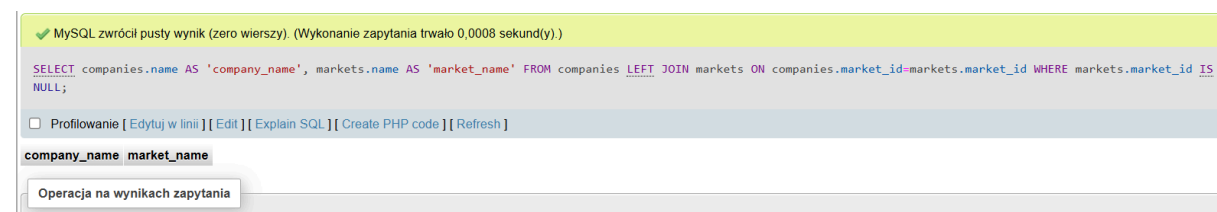
Description: Ensure that the companies table does not contain records with a *market_id* that does not exist in the *markets* table.

✅ **Test passed**

Actual result: The query did not return any records violating the condition. All records in the *companies* table have valid *market_id* values corresponding to the *markets* table.

Expected result: All companies are assigned to a valid market.

```
SELECT companies.name AS 'company_name', markets.name AS 'market_name'
FROM companies
LEFT JOIN markets ON companies.market_id=markets.market_id
WHERE markets.market_id IS NULL;
```



TEST CASE 2

Title: Verification of company ticker uniqueness

Objective: Check whether stock symbols (tickers) are unique.

Description: Identify any duplicates in the *ticker* column of the *companies* table.

✅ **Test passed**

Actual result: The query did not return any records — all tickers appear only once, confirming their uniqueness.

Expected result: No duplicate tickers.

```
SELECT ticker, COUNT(*) AS "number_of_occurrences"
FROM `companies`
GROUP BY ticker
HAVING COUNT(*) > 1;
```

✅ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0007 sekund(y).)

```
SELECT ticker, COUNT(*) AS "number_of_occurrences" FROM `companies` GROUP BY ticker HAVING COUNT(*) > 1;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

ticker	number_of_occurrences
--------	-----------------------

TEST CASE 3

Title: Verification of quote dates

Objective: Ensure that quote dates (*trade_date*) do not exceed the current date.

Description: Check whether the *dailyquotes* table contains any records with future dates.

✅ **Test passed**

Actual result: The query did not return any records — all quote dates are equal to or earlier than the current date.

Expected result: All quotes have dates that do not exceed the current day.

```
SELECT *
FROM dailyquotes
WHERE trade_date > CURRENT_DATE;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0014 sekund(y).)

```
SELECT * FROM dailyquotes WHERE trade_date > CURRENT_DATE;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

quote_id	company_id	trade_date	open_price	close_price	high_price	low_price	volume
----------	------------	------------	------------	-------------	------------	-----------	--------

Operacja na wynikach zapytania

TEST CASE 4

Title: Validation of price ranges in quotes

Objective: Ensure that *high_price*, *low_price*, *open_price*, and *close_price* values are logically correct.

Description: Check that *high_price* $\geq$ *low_price* and all prices are positive.

✓ **Test passed**

Actual result: The query did not return any records — all price values are positive and satisfy the condition *high_price* $\geq$ *low_price*.

Expected result: No incorrect price values.

```
SELECT high_price, low_price, open_price, close_price
FROM dailyquotes
WHERE high_price < low_price
      OR high_price < 0
      OR low_price < 0
      OR open_price < 0
      OR close_price < 0;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0070 sekund(y).)

```
SELECT high_price, low_price, open_price, close_price FROM dailyquotes WHERE high_price < low_price OR high_price < 0 OR low_price < 0 OR open_price < 0 OR close_price < 0;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

high_price	low_price	open_price	close_price
------------	-----------	------------	-------------

Operacja na wynikach zapytania

TEST CASE 5

Title: Verification of quote data completeness

Objective: Check whether each company has corresponding quote data.

Description: Ensure that the *companies* table does not contain companies without related records in the *dailyquotes* table.

✓ **Test passed**

Actual result: The query did not return any records — every company has at least one quote in the *dailyquotes* table.

Expected result: Every company has at least one quote.

```
SELECT companies.name, dailyquotes.*
FROM companies
LEFT JOIN dailyquotes ON companies.company_id = dailyquotes.company_id
WHERE dailyquotes.company_id IS NULL;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0005 sekund(y).)

`SELECT companies.name, dailyquotes.* FROM companies LEFT JOIN dailyquotes ON companies.company_id = dailyquotes.company_id WHERE dailyquotes.company_id IS NULL;`

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

name	quote_id	company_id	trade_date	open_price	close_price	high_price	low_price	volume
------	----------	------------	------------	------------	-------------	------------	-----------	--------

Operacja na wynikach zapytania

TEST CASE 6

Title: Verification of transaction accuracy

Objective: Ensure that all transactions (*trades*) relate to existing companies and dates with quotes.

Description: Check whether there are any transactions that do not have a corresponding quote in *dailyquotes* for the same company and date.

✓ **Test passed**

Actual result: The query did not return any records — all transactions have corresponding quotes in the *dailyquotes* table for the same company and date.

Expected result: All transactions are linked to valid quotes.

```
SELECT DISTINCT trades.trade_id, companies.name, dailyquotes.trade_date
FROM trades
LEFT JOIN companies ON trades.company_id = companies.company_id
LEFT JOIN dailyquotes ON trades.company_id = dailyquotes.company_id
WHERE companies.company_id IS NULL
OR dailyquotes.company_id IS NULL;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0068 sekund(y).)

`SELECT DISTINCT trades.trade_id, companies.name, dailyquotes.trade_date FROM trades LEFT JOIN companies ON trades.company_id=companies.company_id LEFT JOIN dailyquotes ON trades.company_id=dailyquotes.company_id WHERE companies.company_id IS NULL OR dailyquotes.company_id IS NULL;`

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

trade_id	name	trade_date
----------	------	------------

Operacja na wynikach zapytania

TEST CASE 7

Title: Verification of user and subscription data integrity

Objective: Ensure that each subscription relates to an existing user and market.

Description: Identify records in subscriptions that reference non-existent *user_id* or *market_id*.

✅ **Test passed**

Actual result: The query did not return any records — all subscriptions are linked to valid users and markets.

Expected result: No incorrect subscription relationships.

```
SELECT subscriptions.subscription_id, users.user_id, markets.market_id
FROM subscriptions
LEFT JOIN users ON subscriptions.user_id=users.user_id
LEFT JOIN markets ON subscriptions.market_id=markets.market_id
WHERE users.user_id IS NULL OR markets.market_id IS NULL;
```

✔ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0.0005 sekund(y).)

```
SELECT subscriptions.subscription_id, users.user_id, markets.market_id FROM subscriptions LEFT JOIN users ON subscriptions.user_id=users.user_id LEFT JOIN markets ON subscriptions.market_id=markets.market_id WHERE users.user_id IS NULL OR markets.market_id IS NULL;
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

subscription_id	user_id	market_id
-----------------	---------	-----------

Operacja na wynikach zapytania

TEST CASE 8

Title: Validation of active subscriptions

Objective: Verify how many users have an active subscription (without an end date or with a future end date).

Description: Count active subscriptions for each market.

✅ **Test passed**

Actual result: The query returned a list of all markets along with the number of users with an active subscription. All data matched expectations.

Expected result: A list of markets and the number of active users.

```
SELECT COUNT(subscriptions.user_id) AS 'active subscriptions', markets.name
FROM subscriptions
INNER JOIN markets ON subscriptions.market_id = markets.market_id
WHERE end_date > CURRENT_DATE OR end_date IS NULL
GROUP BY markets.name;
```

✔ Pokazano wiersze 0 - 2 (3 total, Wykonanie zapytania trwało 0,0072 sekund(y).)

```
SELECT COUNT(subscriptions.user_id) AS 'active subscriptions', markets.name FROM subscriptions INNER JOIN markets ON subscriptions.market_id = markets.market_id WHERE end_date > CURRENT_DATE OR end_date IS NULL GROUP BY markets.name;
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Liczba wierszy: 25 | Filter rows: Przeszukaj tę tabelę

Extra options

active subscriptions	name
1	Euromarket Amsterdam
1	Euromarket Lisbon
1	Euromarket Paris

TEST CASE 9

Title: Verification of user email accuracy

Objective: Ensure that all email addresses have a valid format.

Description: Check whether the email column contains any records without the @ symbol or a dot (.).

✔ **Test passed**

Actual result: The query returned 0 records — all email addresses contain both the @ symbol and a dot.

Expected result: No invalid email addresses.

```
SELECT email
FROM `users`
WHERE email NOT LIKE '%@%' AND email NOT LIKE '%.%';
```

✔ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0006 sekund(y).)

```
SELECT email FROM `users` WHERE email NOT LIKE '%@%' OR email NOT LIKE '%.%';
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

email

TEST CASE 10

Title: Verification of subscription dates

Objective: Ensure that the end date (*end_date*) is not earlier than the start date (*start_date*).

Description: Identify cases where *end_date* < *start_date*.

✔ **Test passed**

Actual result: The query did not return any records — all subscriptions have a valid date range.

Expected result: All subscriptions have logically correct dates.

```
SELECT start_date, end_date
FROM `subscriptions`
WHERE start_date > end_date;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0003 sekund(y).)

```
SELECT start_date, end_date FROM `subscriptions` WHERE start_date > end_date;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

start_date	end_date
------------	----------

TEST CASE 11

Title: Analysis of companies' average closing prices

Objective: Calculate the average closing price (*close_price*) for each company.

Description: Determine which companies have the highest and lowest average closing prices.

✓ **Test passed**

Actual result: The query returned a list of all companies along with their average closing prices. The data was correctly sorted from the highest to the lowest average value.

Expected result: A report showing the average closing price for each company, sorted in descending order.

```
SELECT AVG(dailyquotes.close_price) AS 'avg_close_price', companies.name
FROM dailyquotes
INNER JOIN companies ON dailyquotes.company_id = companies.company_id
GROUP BY companies.name
ORDER BY avg_close_price DESC;
```

✓ Pokazano wiersze 0 - 3 (4 total, Wykonanie zapytania trwało 0,0004 sekund(y).)

```
SELECT AVG(dailyquotes.close_price) AS 'avg_close_price', companies.name FROM dailyquotes INNER JOIN companies ON dailyquotes.company_id = companies.company_id GROUP BY companies.name ORDER BY avg_close_price DESC;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Liczba wierszy: 25 | Filter rows:

Extra options

avg_close_price	name
835.500000	LVMH
61.050000	TotalEnergies
19.000000	Philips
13.500000	Galp Energia

TEST CASE 12

Title: Verification of closing prices within minimum and maximum range

Objective: Ensure that *close_price* falls within the *low_price* – *high_price* range.

Description: Identify records in *dailyquotes* where *close_price* is less than *low_price* or greater than *high_price*.

✅ **Test passed**

Actual result: The query did not return any records — all closing prices fall within the minimum and maximum price range.

Expected result: All closing prices fall within the minimum and maximum price range.

```
SELECT quote_id, company_id, trade_date, close_price, high_price, low_price
FROM dailyquotes
WHERE close_price > high_price OR close_price < low_price;
```

✅ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0003 sekund(y).)

```
SELECT quote_id, company_id, trade_date, close_price, high_price, low_price FROM dailyquotes WHERE close_price > high_price OR close_price < low_price;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

quote_id	company_id	trade_date	close_price	high_price	low_price
----------	------------	------------	-------------	------------	-----------